

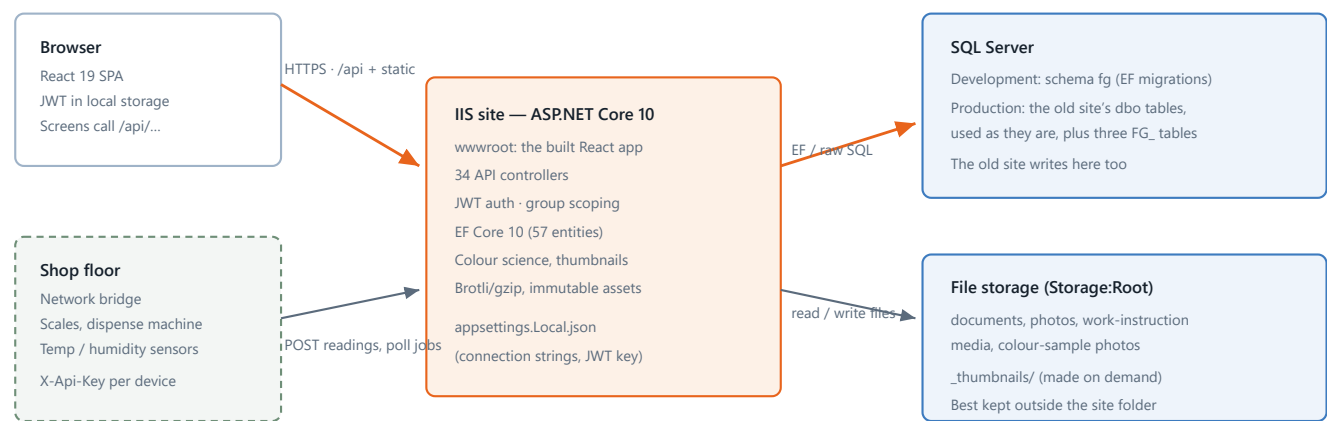
Finish Genius — Architecture

Where the data comes from, where it goes, and what holds it

Written for developers and whoever runs the servers · Companion to “Automated Formulation & Wood.pdf” in this folder, with the colour-matching feature described in its own section.

1. The shape of the system

Finish Genius is **one website**. A single ASP.NET Core 10 application serves both the JSON API and the React app that consumes it: the front end is built by Vite into the API’s `wwwroot`, so there is one process, one port, one certificate and no cross-origin setup. It talks to **SQL Server** and to a folder of uploaded files.



Everything a browser needs comes from one site; devices post to the same API with their own key.

2. Two databases, chosen per sign-in

This is the part that surprises people. The app can work against **two different databases** and decides which on every request:

Database	What it is	How the app uses it
Development	Finish Genius’s own schema, <code>fg</code>	Created and updated by EF Core migrations, applied at startup. This is the shape the app was designed for.
Production	The old site’s database, still live and still written to by the old site	Used as it is . The same entities are mapped onto the old <code>dbo</code> tables; nothing is migrated, copied or renamed. Reads go through hand-written SQL (<code>ToSqlQuery</code>), writes go to the real tables.

Which one a session uses is decided at sign-in, put into the JWT as a `db` claim, and read back on every request (`DatabaseSelector`). `Database:Use` forces one; otherwise the address the browser used decides, via `Database:ProductionHosts`.

Consequence to keep in mind: on Production the app is a guest in someone else's database. It never migrates it, and the only tables it adds are prefixed `FG_`: `FG_LoginAudit` (sign-in log), `FG_PageAccess` (page switches) and `FG_ColorSamples` (colour library). Each is created on first use. Legacy tables have almost no indexes, so queries there are written run-by-run rather than as big joins.

3. A request, end to end

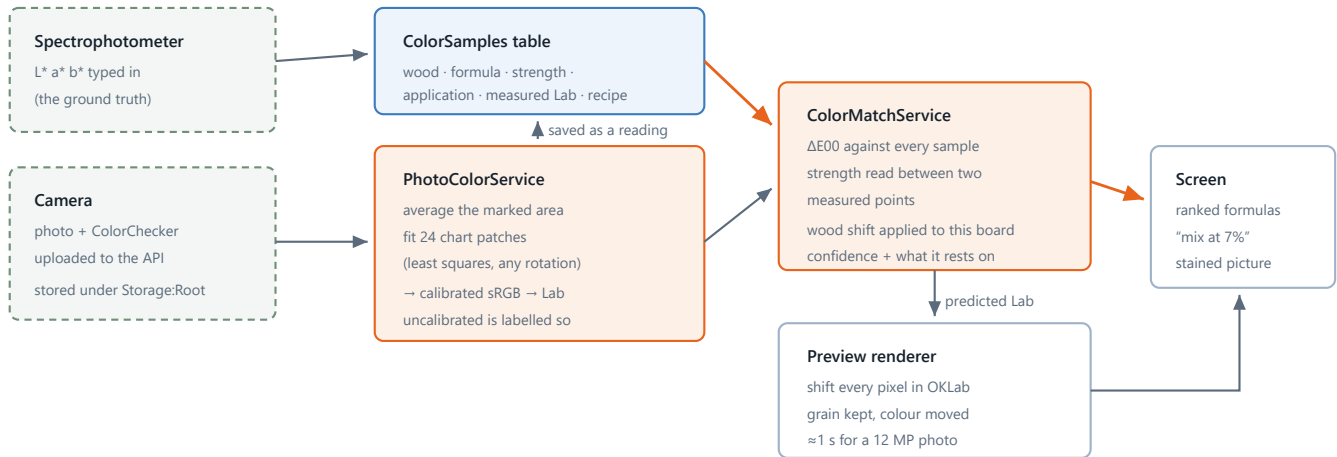
1. **Browser** loads `index.html` (never cached) and the hashed files under `/assets` (cached for a year, immutable). The page's own code is fetched while the session check is still in flight.
2. **Sign-in** posts to `/api/auth/login`. The reply is a JWT carrying the user id, roles, group and the database key. The browser keeps it and sends it as `Authorization: Bearer ...`.
3. **Every API call** passes through: JWT validation → `DatabaseSelector` (which database) → `PageAccessMiddleware` (has the owner switched this page off?) → the controller.
4. **The controller** resolves the caller's groups (`CurrentUser.EnsureGroupAsync`) — everything in the app belongs to a group, and that check is what keeps one customer's data out of another's — then reads or writes through EF Core and returns a DTO, never an entity.
5. **Responses** are compressed (brotli/gzip). Changes are written to the audit log in the same transaction as the change itself.

Where each kind of data lives

Data	Comes from	Stored in	Read back by
Groups, users, roles	Administration screens	<code>fg.Groups/Users</code> · legacy <code>dbo.Groups</code> , <code>dbo.[User]</code>	Every screen (group scoping)
Materials, categories, vendors, inventory	Equipment & Materials, Import	<code>fg.Materials</code> · legacy <code>dbo.Materials</code> and batch tables	Formulas, process steps, colour matching
Formulas and their ingredients	Formulas workspace	legacy <code>dbo.Materials</code> (Discriminator 'Formulation') + <code>FormulationMaterials</code>	Dispensing, pricing, colour matching
Process steps, schedules, pricing	Processes screens	<code>FinishingSteps</code> , <code>FinishingSchedules</code> and their detail tables	My Work, Material Quantities, Pricing
Shop-floor runs (checklists, defects, adders)	My Work, on the floor	<code>MyWorkExecution*</code> tables	Dashboard, Process data
Device telemetry	Sensors → <code>POST /api/devices/ingest</code>	<code>dbo.DeviceMetrics</code> (11.7 M rows on Production)	Dashboard → Devices charts
Documents, photos, media	Uploads	File storage + link tables	<code>GET /api/files/{path}</code>
Colour samples	Color Matching	<code>fg.ColorSamples</code> / <code>dbo.FG_ColorSamples</code>	Matching, prediction, preview
Audit trail	Every create/update/delete	<code>fg.AuditLogs</code> / <code>dbo.History</code>	History modals, Group activity
Page access switches	Owner account	<code>FG_PageAccess</code> (database, not a file)	Menu, routes and APIs
Connection strings, JWT key, owner	Set by hand on the server	<code>appsettings.Local.json</code>	Startup only
Upload folders	System Settings screen	<code>App_Data/storage-settings.json</code>	Startup + file serving

4. Colour matching — the data path

This is the feature the requirements in this folder describe. Nothing about it is a black box: every answer is computed from rows somebody recorded.



Readings come from a device or a calibrated photograph; everything downstream is arithmetic over recorded samples.

The pieces

Part	File	What it owns
Colour maths	<code>Services/ColorScience.cs</code>	sRGB ↔ Lab ↔ OKLab, ΔE76, ΔE00 (checked against the published reference data)
Chart calibration	<code>Services/ColorChecker.cs</code>	The 24 reference patches and the least-squares fit that corrects a photo
Photo reading, preview	<code>Services/PhotoColorService.cs</code>	Region averaging, grey-card and chart correction, region suggestion, the OKLab render
Matching and prediction	<code>Services/ColorMatchService.cs</code>	Ranking, strength interpolation, wood correction, confidence and its wording
API	<code>Controllers/ColorMatchingController.cs</code>	Samples CRUD, photo upload, read-photo, suggest-regions, match, predict, preview
Screens	<code>frontend/src/pages/colors/</code>	Sample Library, Match a Colour, Stain Preview, the region picker

The rule the design follows: a prediction may only be arithmetic over measured samples, and it must say how many it used. There is no model, nothing is extrapolated beyond the strengths that were mixed, and a formula with no samples produces a sentence saying so rather than a number.

5. Data coming in from the shop floor

Two device paths exist, both authenticated by a per-device key rather than a user session:

- **Telemetry** — a sensor or bridge posts to `POST /api/devices/ingest` with `X-Api-Key`. Readings land in the device metrics table and appear on Dashboard → Devices. An empty batch is a heartbeat and only updates “last seen”.
- **Jobs** — the network bridge polls `GET /api/bridge/commands` for dispensing, purging, scale reads and label prints, and posts results back. This replaces the old site’s SignalR push, so the bridge needs no inbound port.

The keys are shown (and can be regenerated) on Dashboard → Devices. Regenerating one stops the old key immediately.

6. Files

Uploads never live in the database. `Storage:Root` — normally set on the System Settings screen to a folder **outside the site**, e.g. `D:\FinishGeniusData` — holds documents, photos, work-instruction media and colour-sample photographs, under `<kind>/<groupId>/`. Files of the old site can be read in place through `Storage:LegacyRoot`.

They are served by `GET /api/files/{path}`, which checks the caller belongs to the owning group. Because an `<img>` cannot send an Authorization header, those URLs carry `?access_token=`. Add `&w=600` and the API returns a cached thumbnail instead of an 8 MB original — that is what keeps the photo gallery quick.

7. What must survive a deployment

A new build replaces the site’s files. These belong to the server and are never in a package:

Keep	Holds
<code>appsettings.Local.json</code>	Connection strings, JWT key, owner account
<code>App_Data\storage-settings.json</code>	The upload folders chosen in System Settings
<code>App_Data\uploads\</code>	Uploaded files, when <code>Storage:Root</code> is left empty
<code>logs\</code>	stdout logs

`deploy\Install-IIS.ps1` keeps all four, and empties `wwwroot` before copying so the previous build’s scripts cannot linger. **Page Access is deliberately not a file** — it lives in the database precisely so a deployment cannot reset it.

8. Performance, and why it is written this way

- **Payloads are compressed** (brotli/gzip). A process-step builder response is 1.35 MB of JSON and 311 KB on the wire.
- **Hashed assets are immutable**, `index.html` never cached: a repeat visit fetches no JavaScript at all.

- **The session check does not block the page.** The last known user is cached beside the token, so a screen asks for its own data at the same time as `/auth/me`, not a round trip later.
- **Legacy queries are shaped for a database with no indexes.** Runs are read one at a time; screens that could open hundreds of them ask for a fixed number and say so on screen. A join across those tables once filled Production's tempdb, which is why the rule exists.
- **Heavy pictures are resized once** and cached on disk.

9. Access control

- **Roles** (System Admin, Group Admin, FG Pro, FG Pro Plus, Support) decide what a user may reach; controllers carry the role list, and the menu is filtered to match.
- **Groups** are the tenancy boundary: every list takes a group and every record is checked against the caller's groups before it is read or written.
- **Page Access** lets the owner account switch pages and tabs off per role. It is enforced on the API as well as in the menu, so hiding a page really does close its endpoints.
- **The owner account** lives in configuration, not in any database, has no user record and cannot change business data — it exists to configure access and to see the sign-in log.

10. Build and deploy

1. `build.ps1` — builds the React app into `wwwroot`, publishes the API, writes an idempotent SQL script and zips the result.
2. Copy the zip to the server, extract, run `deploy\Install-IIS.ps1`.
3. The app applies its own migrations to the development database at startup; the Production database is never migrated, and the tables it adds are created on first use.

Related documents in the repository: `docs/DEPLOYMENT.md`, `docs/CONVENTIONS.md` (how to add a module), `docs/COLOR_MATCHING.md` (what the colour feature does and does not do), and `database/FG_*.sql` for the tables added to the old database.